arXiv:2606.02486v2 [cs.RO] 3 Oct 2026

Intercepting the Future: Latent-Space Predictive World Model for Dynamic VLA Manipulation

Shahram Najam Syed[1] snsyed@andrew.cmu.edu
Arthur Jakobsson[1]* ajakobss@andrew.cmu.edu
Haoran Hao[1]* haoranha@andrew.cmu.edu

Jeffrey Ichnowski[1] jichnows@andrew.cmu.edu

[1]Robotics Institute, Carnegie Mellon University, Pittsburgh, USA

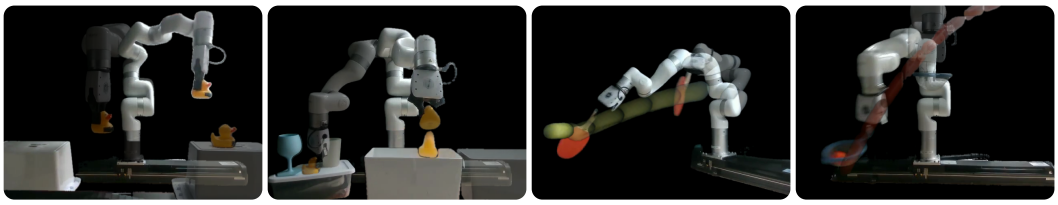


Figure 1: AHEAD on four real-world dynamic manipulation tasks. Translucent overlays show object trajectories and transient robot positions; solid silhouettes show the final manipulation pose.

Abstract: Vision-Language-Action (VLA) models generalize across static manipulation but fail when objects move during task execution. Policies that act on the current observation assume the scene is stationary until execution, so at any non-trivial object speed the resulting latency exceeds the time available to grasp. We close this gap with AHEAD (Anticipatory Horizon Extrapolation with Adaptive Dynamics), a predict-then-act wrapper that augments a frozen VLA with a motion-aware latent world model. A small world model trained on manipulation video forecasts future patch tokens in the VLA's feature space, conditioned on per-token velocity and acceleration from optical flow. A language-and-motion saliency mask concentrates prediction on task-relevant patches, and the model rolls forward for an adaptive horizon, halting when prediction uncertainty crosses a threshold. The frozen action decoder then receives the predicted future tokens in place of the current ones. AHEAD adds 4.9M parameters to a frozen 7B OpenVLA and reaches 79 to 97% success across 20 dynamic simulation scenarios where the strongest baseline reaches 31 to 58%. On a physical UFactory xArm 7, AHEAD succeeds on $29/30$ to $30/30$ on three conveyor and rolling-ball tasks, $23/30$ on paddle interception, and $19/30$ on projectile catching where every baseline scores $0/30$.

Keywords: Vision-Language-Action Models, World Models, Manipulation

1 Introduction

> *"I skate to where the puck is going to be, not where it has been." — Wayne Gretzky*

Vision-Language-Action (VLA) models [1, 2, 3, 4] have brought general-purpose manipulation policies within reach, but they remain confined to scenes where the world holds still. The moment

*Equal contribution.

10th Conference on Robot Learning (CoRL 2026), Austin TX, USA.

objects move, on conveyors, in handovers, after release, or under gravity, current VLAs fail at a regime humans handle through anticipatory internal models that track object velocity, plan toward predicted intercept points, and update from contact feedback [5, 6]. Policies that act on the current observation assume the scene is stationary until execution, so at any non-trivial object speed the resulting latency exceeds the time available to grasp. Prior work has pursued two directions to close this gap. Reactive policies [7, 8] dual-system VLAs [9, 10], and recent concurrent work [11, 12] shorten the perception-to-action loop through faster control and architectural changes. This shortens the stale window without removing it, and as object speed grows, the residual latency consumes a larger fraction of the reaction window regardless of how fast the policy runs. World model approaches [13, 14, 15, 16] learn forward dynamics and plan through imagined rollouts, but existing formulations roll the entire scene over a fixed horizon and operate either in expensive pixel space or require joint retraining. Neither line addresses which parts of the scene need prediction, how far ahead, or at what latency. We introduce AHEAD (Anticipatory Horizon Extrapolation with Adaptive Dynamics), a predict-then-act wrapper that augments a frozen VLA with a motion-aware latent world model in the VLA's feature space. The world model rolls forward only the patches identified as task-relevant by language conditioning and as independently moving by per-patch optical flow, halts each rollout when prediction uncertainty crosses a threshold, and feeds the predicted future tokens to the frozen action decoder. We evaluate AHEAD on 20 dynamic simulation scenarios and five physical xArm 7 tasks. In simulation, AHEAD reaches 79 to 97% success against 31 to 58% for the strongest baseline. On the physical robot, it succeeds on $19/30$ projectile catches with a launcher 2 m from the robot, where every baseline scores $0/30$. This paper contributes:

- A predict-then-act wrapper for frozen VLAs that enables dynamic-object manipulation without retraining the underlying VLA.
- Adaptive compute allocation along both spatial and temporal axes that keeps prediction inside the reactive latency budget, where language-and-motion saliency selects which patches to predict and uncertainty-driven halting determines how far to roll forward.
- Explicit kinematic conditioning that propagates per-token velocity and acceleration analytically across rollout steps, extending the world model from constant-velocity to acceleration regimes without learning second-order physics from data.

2 Related Work

Vision-Language-Action Models. VLAs unify visual perception, language understanding, and motor control in a single end-to-end architecture, from RT-2's [3] co-fine-tuning of vision-language models with robotic trajectories, to OpenVLA's [1] open-source 7B model, Octo's [2] cross-embodiment transformer, π_0's [4] flow-matching action head, and GR00T N1's [17] dual-system humanoid controller. These methods act on the current observation with no mechanism for anticipating scene evolution. Dual-system designs such as HiRT [9] and RoboDual [10] pair a slow VLM with a fast low-level policy, which shortens the stale window without removing it. AHEAD addresses this gap by wrapping a frozen VLA with a world model that predicts task-relevant future states.

World Models in Latent Space. World models support planning through imagined rollouts. The Dreamer family [18, 19, 13] optimizes policies entirely in imagination but requires joint training of policy and dynamics, which is incompatible with a frozen pretrained VLA. TD-MPC2 [20] learns latent dynamics for online MPC, with cost scaling in the number of sampled trajectories. Generative video models [14, 21] and autoregressive variants integrated into VLAs [15] produce rollouts whose inference exceeds the reactive latency budget. The closest precursor is LaDi-WM [16], a latent diffusion world model that, like AHEAD, predicts in pretrained feature space. AHEAD differs by replacing latent diffusion with conditional flow matching, conditioning on explicit per-token kinematic state from optical flow rather than asking the model to infer motion, and rolling forward selectively with adaptive horizon halting where LaDi-WM uses the full state at a fixed horizon. To

our knowledge, AHEAD is the first latent robotic world model to allocate compute adaptively along both spatial and temporal axes.

Model-Predictive Control with Learned Models. Sampling-based MPC with learned dynamics [22, 23, 24] evaluates K candidate trajectories per timestep and selects or weights them by predicted return; inference cost scales with K. Classical MPC [25] solves a single optimization over a fixed horizon but requires a differentiable model that is hard to combine with learned VLA action heads. AHEAD differs from both. The world model rolls forward once to a single predicted future state which feeds directly to the action decoder, with $S{=}5$ samples used for uncertainty estimation rather than action selection.

Reactive and Predictive Dynamic Manipulation. Reactive policies [7, 8] reduce observation staleness through action chunking but react to where an object is rather than where it will be; classical interception methods [26, 27] rely on hand-designed state representations without large-scale pretraining. Concurrent work approaches dynamic VLAs through faster inference (DynamicVLA [11], VLASH [12]) or larger world models for offline evaluation (Ctrl-World [28], VLAW [29]); both accelerate the loop rather than predicting ahead. AHEAD explicitly predicts where the object will be and acts on that prediction, decoupling effective planning horizon from VLA perception-execution latency.

Video Pretraining for World Models. Large-scale video offers object motion dynamics that bootstrap world models without robot demonstration collection [30, 31, 32]. A natural candidate is Ego4D [33], whose 3,600+ hours of egocentric video are dominated by outdoor walking and whole-body dynamics that differ from fixed-camera tabletop manipulation. We instead draw from a curated manipulation-focused corpus combining EPIC-Kitchens [34], Something-Something V2 [35], DROID [36], and Bridge V2 [37], whose flow statistics match the deployment setting. Because AHEAD's world model operates in VLA feature space rather than pixel space, any manipulation-relevant motion video provides useful training signal.

3 Problem Statement

Setting. Consider a robot operating in an environment where objects may move independently of the robot's actions. At each time step t, the robot receives a monocular RGB observation o_t and a natural-language task instruction ℓ. A frozen pretrained VLA provides a vision encoder ϕ that maps observations to N patch tokens $v_t = \phi(o_t) \in \mathbb{R}^{N \times d}$, where d is the patch token dimension. The VLA also provides a language encoder that produces instruction embeddings e_ℓ, and an action decoder π that generates a motor command $a_t = \pi(v_t, e_\ell) \in \mathcal{A}$. The scene state evolves from two sources, robot-induced change from a_t and independent change from external agents or physical processes. Policies that act on the current observation implicitly assume the scene remains static until execution, so when independent change is present, actions planned at time t target an outdated scene state.

Objective. We aim to augment the frozen VLA with the ability to anticipate independent scene change, so that the action decoder receives an estimate of future scene state in place of the stale current observation. The augmentation should preserve the frozen VLA's pretrained capabilities and operate within the real-time control loop, leaving the underlying action decoder and language grounding unchanged.

Desiderata. An effective predict-then-act loop must satisfy two desiderata. First, **scene-adaptive prediction horizon**. The realized horizon should match per-scene predictability rather than being fixed. Linear, low-uncertainty motion admits long horizons that anticipate contact well in advance, whereas chaotic or post-collision motion admits only short horizons before compounding prediction error dominates, so a solution must expose some signal of per-scene prediction reliability. Second, **real-time execution**. The full loop from observation to predicted future to action must complete

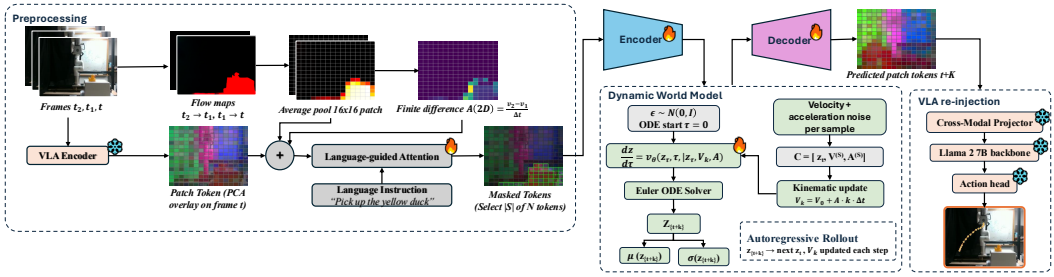


Figure 2: **AHEAD architecture.** RAFT optical flow [38] provides per-patch velocity and acceleration. A Language-and-motion saliency mask selects a task-relevant subset $\mathcal{S}$ of N patch tokens, which are encoded, rolled forward by a conditional flow-matching world model with analytical kinematic conditioning $V_k = V_0 + A \cdot k \cdot \Delta t$, decoded, and spliced back into the full token grid for the frozen action decoder. Flame icons mark trained components ($\sim$4.9M parameters total); snowflakes mark components frozen from OpenVLA. Figure 3 in Appendix A shows the full breakdown including autoregressive rollout, adaptive halting, and feature alignment.

within a latency budget $\Delta t_{\max}$, since predictions arriving after $\Delta t_{\max}$ are stale by the time they reach the action decoder.

Scope. This work targets *dynamic scenes*, settings in which objects move independently of the robot during task execution, including human-initiated object displacement, conveyor transport, and post-contact rolling. The method assumes object motion is well-approximated by constant acceleration (including the zero-acceleration limit of constant velocity) over the prediction horizon. Highly stochastic post-collision dynamics violate this assumption; we evaluate such scenarios as a stress test and report graceful degradation rather than competitive performance (Appendix D.1).

4 Method

AHEAD augments a frozen VLA with a motion-aware latent world model so that the robot acts on predicted future states rather than stale observations. At each time step t, the frozen VLA vision encoder ϕ maps the current RGB observation o_t to N patch tokens $v_t \in \mathbb{R}^{N \times d}$. Standard VLAs feed v_t directly to the action decoder and act on what the world looked like at time t, which is already stale by the time the action executes. AHEAD instead predicts $\tilde{v}_{t+K}$, the patch tokens corresponding to the scene state K steps in the future, and feeds those predicted tokens to the unchanged action decoder (Figure 2). Three observations shape the pipeline. First, most patches in a manipulation scene do not move and do not warrant compute; only patches with task-relevant motion need prediction. Second, the right prediction horizon K depends on the scene; linear motion admits long lookahead, chaotic motion does not. Third, the entire prediction must complete within a reactive manipulation latency budget on the order of 200 ms. These observations motivate spatial compute allocation (Section 4.1), temporal compute allocation (Section 4.3), and the dynamics model choice (Section 4.2). Section 4.4 describes training. Appendix B gives full architectural details, ablation rationale, and hyperparameters.

4.1 Language-and-Motion Saliency

RAFT [38] computes optical flow between three consecutive observations, which patch-level pooling and finite-differencing convert to per-token velocity V and acceleration A (Appendix B.5). We concatenate each patch token with its motion descriptor to form motion-enriched tokens $\tilde{v}_t \in \mathbb{R}^{N \times (d+4)}$. To allocate compute spatially, AHEAD predicts only on tokens that are either language-relevant or independently moving. A learned cross-attention layer attends from $\tilde{v}_t$ to the instruction embeddings e_ℓ, producing per-token language relevance scores $\widetilde{M} \in [0, 1]^N$. Tokens

whose velocity magnitude exceeds a calibrated threshold τ_{flow} receive an elevated relevance floor,

$$M_i = \max\Big(\widetilde{M}_i,\ \alpha_{\text{motion}} \cdot \mathbf{1}[\|V_i\| > \tau_{\text{flow}}]\Big). \tag{1}$$

AHEAD forms the selected index set $\mathcal{S} = \{i : M_i > 0.2\}$, typically yielding 30 to 60 tokens out of $N = 196$ in our experiments. Appendix B.4 gives morphological dilation, threshold calibration, and α_{motion}.

4.2 Flow-Matching World Model with Kinematic Conditioning

The world model operates in VLA feature space (rather than pixels) to align with the action decoder's input distribution and avoid expensive reconstruction. AHEAD uses conditional flow matching [39], which produces high-quality samples in a few Euler steps [40] against 100+ for diffusion, keeping the world model within the per-step latency budget (Appendix B.3). Drawing S samples in parallel yields both a mean prediction and a sample-variance uncertainty estimate at no extra cost. A 4-layer transformer encoder compresses the $|\mathcal{S}|$ saliency-selected, motion-enriched tokens $\tilde{v}_t^{(\mathcal{S})}$ into a compact latent z_t, and the dynamics model rolls forward autoregressively,

$$z_{t+k} \sim p\Big(z_{t+k} \,\Big|\, z_{t+k-1},\ V_k^{(\mathcal{S})},\ A^{(\mathcal{S})}\Big), \quad k = 1, \ldots, K. \tag{2}$$

To avoid asking the world model to learn full physics from data, AHEAD propagates the velocity conditioning analytically across rollout steps under constant-acceleration kinematics,

$$V_k = V_0 + A \cdot k \cdot \Delta t, \tag{3}$$

where V_0 is the initial RAFT-derived velocity and A the finite-differenced acceleration. This captures the dominant trajectory curvature in acceleration-driven scenarios such as gravitational pull and friction decay. An MLP decoder reconstructs predicted patch tokens, which are spliced back into the full grid (unselected patches pass through unchanged) before being passed to the frozen action decoder, which produces $a_t = \pi(\hat{v}_{t+K}, e_\ell)$. The spliced grid is a functional surrogate for the future observation rather than a visual reconstruction (Appendix B.1). Appendix B.1 details architecture dimensions; Appendix B.2 compares against diffusion and dropout-based GRU alternatives.

4.3 Adaptive Horizon Halting

To allocate compute temporally, AHEAD halts each rollout when prediction uncertainty crosses a threshold. The S trajectory samples drawn at each step (Section 4.2) provide a per-step scene-level uncertainty estimate as their mean per-token variance,

$$\bar{u}_{t+k} = \frac{1}{|\mathcal{S}|} \sum_{i \in \mathcal{S}} \frac{1}{S} \sum_{j=1}^{S} \big\| z_{t+k}^{(j)}[i] - \bar{z}_{t+k}[i] \big\|^2, \tag{4}$$

where $z_{t+k}^{(j)}$ is the j-th sample and $\bar{z}_{t+k}$ the sample mean. The rollout halts at the first step K satisfying $\bar{u}_{t+K} > \tau_u$, or at maximum horizon $K_{\max}$ otherwise. Linear, low-uncertainty motion runs longer; chaotic or post-collision motion halts earlier. In experiments we set $K_{\max} = 10$ and τ_u to the 90th percentile of $\bar{u}$ on the training set.

4.4 Training

The world model is trained in two phases. Pretraining on a corpus of manipulation video establishes a prior over object dynamics in VLA feature space, since any video containing manipulation-relevant motion provides useful training signal once features are extracted by ϕ. Fine-tuning on ≈ 200 in-domain xArm 7 trajectories closes the domain gap to the deployment setting. The dynamics model uses the conditional flow matching objective; the encoder and decoder use MSE reconstruction over the rollout horizon. The feature alignment layer is trained separately. Appendix B.8 specifies the pretraining corpus, curriculum, loss functions, and hyperparameters.

5 Experiments

We evaluate AHEAD in simulation across 20 dynamic-object manipulation scenarios and on a physical UFactory xArm 7 across five dynamic tasks. Ablations isolate the contribution of each component.

5.1 Experimental Setup

Simulation. We use custom MuJoCo [41] environments built around a Franka Emika Panda [42] arm spanning four motion categories: constant-velocity transport (conveyor, beam, pole push with red cup), gravity-driven motion (rolling ball), reactive contact (air hockey, ballistic catching), and chaotic post-collision dynamics (multi-deflection pinball, occlusion deflection, plinko drop). Each scenario is procedurally generated with randomized object positions, motion parameters, and lighting. Appendix C.1 gives scene specifications and per-scenario success criteria.

Physical platform. A UFactory xArm 7 with a fixed third-person RGB camera at 224×224 and a parallel-jaw gripper, running on a single NVIDIA RTX 4090 at 171 ms per action step, evaluated on five tasks: (i) static object onto moving conveyor box, (ii) static box receiving moving object, (iii) projectile interception with a ping-pong paddle, (iv) stopping a rolling ball, and (v) catching a projectile from a mechanical launcher. A speed-sensitivity sweep varies conveyor speed from 0 to 25 cm/s. Appendix C.2 gives hardware details.

Baselines. Six learned baselines span the relevant axes of dynamic manipulation: Open-loop VLA and Retargeting VLA (closed-loop) [1]; VLA + Fast Replan [43] with action chunking; Realtime ACT [7] and Streaming Diffusion Policy [8] for chunked real-time execution; and DreamVLA [44], a world-model-based VLA that predicts dynamic regions and depth before generating actions. Two motion-informed baselines feed the same frozen OpenVLA. Track-and-Intercept is a classical tracker that extrapolates the object centroid under a constant-acceleration fit over AHEAD's realized horizon and warps the frame so the object appears at its predicted location, a pixel-space analogue of our token splice. LaDi-WM [16] is retrained in OpenVLA feature space. Appendix C.3 gives per-baseline hyperparameters and training data.

Protocol and configuration. In simulation, each cell reports mean $\pm$ standard error over 5 training seeds with 100 rollouts per seed (500 rollouts per cell). On the physical robot, each task reports successes out of 30 attempts (10 per speed point for the speed sweep). All AHEAD results use: frozen OpenVLA, RAFT [38] optical flow, kinematic and acceleration-conditioned velocity model, language-and-motion saliency masking, 4-layer encoder, $S = 5$ flow-matching samples with 5 Euler steps, $K_{\max} = 10$, and τ_u at the 90th percentile of $\bar{u}$ on the training set. Realized horizon averages 3 to 5 steps. Full hyperparameters in Table 8.

5.2 Simulation Results

We evaluate three aspects of dynamic-object manipulation in simulation: success across constant-velocity and acceleration regimes (Table 1), robustness to object speed (Table 2), and generalization to complex scenarios with multi-object scenes, occlusion, and mid-flight trajectory changes (Table 3).

Constant velocity and acceleration. Table 1 shows AHEAD outperforming all baselines on every scenario in both regimes, with the largest gap on acceleration-regime scenarios where the strongest baseline (DreamVLA) reaches 30 to 48% while AHEAD stays above 87%. Performance is also stable across regimes (within 10 percentage points), while baselines degrade substantially. Inspecting failure modes, baselines without explicit prediction (Retargeting VLA, VLA + Fast Replan, Realtime ACT) most often fail by grasping where the object was rather than where it will be; the gripper closes on empty space behind the moving object. The explicit kinematic update (Eq. 3) lets AHEAD target the predicted intercept point instead.

Table 1: Task success rate (%) on dynamic scenes across constant-velocity and acceleration/deceleration regimes. The two rows below the first rule are motion-informed baselines that feed the same frozen OpenVLA. Mean ± standard error over 5 seeds with 100 rollouts each.

	Constant velocity				Acceleration / deceleration			
Method	**Conv.**	**Beam**	**Pole**	**Roll**	**Conv.**	**Beam**	**Pole**	**Roll (g.)**
Open-loop VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.4 ± 0.6	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	13.7 ± 3.8	22.3 ± 4.0	13.3 ± 3.2	7.7 ± 2.9	1.0 ± 1.4	10.7 ± 3.1	1.0 ± 1.4	0.6 ± 1.1
VLA + Fast Replan	24.0 ± 4.6	32.7 ± 4.2	22.3 ± 3.7	16.7 ± 4.0	22.3 ± 4.2	31.3 ± 4.6	21.7 ± 4.1	16.3 ± 3.7
Realtime ACT	51.7 ± 5.1	60.3 ± 4.7	48.7 ± 4.9	37.0 ± 5.3	31.7 ± 4.6	41.0 ± 4.9	40.3 ± 4.9	31.0 ± 4.6
Streaming DP	40.7 ± 5.2	50.3 ± 4.8	39.3 ± 4.5	28.7 ± 4.6	21.0 ± 4.1	29.7 ± 4.6	29.0 ± 4.5	19.7 ± 4.0
DreamVLA	58.3 ± 4.7	47.7 ± 5.1	57.3 ± 5.0	46.0 ± 5.2	40.3 ± 4.9	38.7 ± 4.9	48.0 ± 5.0	30.7 ± 4.6
Track-and-Intercept VLA	91.2 ± 2.6	85.4 ± 3.3	88.6 ± 3.0	82.0 ± 3.7	71.4 ± 4.3	74.8 ± 4.1	69.2 ± 4.4	63.6 ± 4.6
LaDi-WM (OpenVLA feat.)	68.4 ± 4.4	61.2 ± 4.7	66.8 ± 4.5	55.4 ± 4.9	49.6 ± 4.8	47.2 ± 4.9	55.8 ± 4.7	38.4 ± 4.6
AHEAD (ours)	$\mathbf{97.3 \pm 1.5}$	$\mathbf{91.7 \pm 2.8}$	$\mathbf{95.0 \pm 2.3}$	$\mathbf{90.0 \pm 3.1}$	$\mathbf{87.7 \pm 3.3}$	$\mathbf{91.0 \pm 3.9}$	$\mathbf{93.7 \pm 6.1}$	$\mathbf{89.0 \pm 0.9}$

Table 2: Task success rate (%) versus conveyor speed on the conveyor + red cup scenario. Mean ± standard error over 5 seeds with 100 rollouts each.

Method	**0 cm/s**	**10 cm/s**	**20 cm/s**	**30 cm/s**	**40 cm/s**
Open-loop VLA	96.4 ± 1.9	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	96.8 ± 1.8	40.6 ± 4.9	31.2 ± 4.4	22.4 ± 4.1	14.6 ± 3.4
Realtime ACT	97.2 ± 1.6	78.4 ± 3.9	64.8 ± 4.4	51.4 ± 4.6	41.0 ± 4.7
Streaming Diffusion Policy	95.4 ± 2.1	31.6 ± 4.4	27.2 ± 4.0	22.4 ± 4.1	18.6 ± 3.7
DreamVLA	96.8 ± 1.8	79.2 ± 3.8	70.4 ± 4.3	60.2 ± 4.4	47.2 ± 4.1
AHEAD (ours)	$\mathbf{97.6 \pm 1.5}$	$\mathbf{97.4 \pm 1.7}$	$\mathbf{97.0 \pm 1.8}$	$\mathbf{96.6 \pm 2.0}$	$\mathbf{95.4 \pm 2.2}$

Robustness to object speed. Table 2 shows AHEAD maintaining over 95% success across the 0 to 40 cm/s sweep while every baseline degrades steadily. The static case (0 cm/s) shows that all methods are well-matched on perception and grasping in isolation; the gap opens once independent object motion is introduced, suggesting the gap is attributable to predictive lookahead rather than confounding perception or action-decoder differences. The 2.2-point residual at 40 cm/s comes from RAFT error at the high-velocity end of the flow regime; the motion-estimator ablation (Appendix E.1) supports this.

Complex scenarios. Table 3 shows AHEAD reaching 79 to 96% on every complex scenario where the strongest baseline reaches at most 60%. The Occlusion (Occ) column is the cleanest demonstration that prediction is qualitatively distinct from reactivity. Every baseline except Realtime ACT scores exactly 0% on Occ, because catching an object that has disappeared behind an occluder requires predicting where it will reappear, which a closed-loop policy looking only at the latest frame cannot solve. AHEAD reaches 79.4%.

Motion-informed baselines. Track-and-Intercept averages 86.8% on constant-velocity scenes against 93.5% for AHEAD, so a kinematic prior alone does much of the work when motion is clean. It reaches 51.2% on Ballistic Catch against 87.4%, as a 2D warp cannot represent scale change and leaves disocclusion holes, and 33.6% on L+MO (language + multiple-objects) against 95.6%, as the tracker cannot resolve the instruction referent. LaDi-WM averages 63.0% on constant-velocity scenes and 18.6% on Occlusion, as full-state prediction at a fixed horizon must infer motion from features alone.

5.3 Physical Robot Experiments

Table 4 shows AHEAD as the strongest method on every physical task, with $30/30$ on conveyor pick-and-place, $30/30$ on rolling ball, and $19/30$ on projectile catching where every baseline scores $0/30$. Transfer is tight on smooth-motion tasks (97.3% sim, $30/30$ real on conveyor) but degrades on contact-physics tasks (paddle hitting 94.6% sim, $23/30$ real; projectile catching 87.4% sim, $19/30$ real), reflecting limits of MuJoCo's collision model, gripper compliance, and camera shutter behavior. The 11 unsuccessful projectile catches break down as follows. Five came from joint-

Table 3: Task success rate (%) on eight complex dynamic scenarios (Air Hockey, Ballistic Catch, Multi-Object, Place in Box, Multi-Balls, Occlusion, Mid-flight deflection, Language + Multi-Object). Full scenario descriptions in Appendix C.1. Mean ± standard error over 5 seeds with 100 rollouts each.

Method	AH	BC	MO	PiB	MB	Occ	Mid	L+MO
Open-loop VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	10.6 ± 3.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
VLA + Fast Replan	0.0 ± 0.0	0.0 ± 0.0	31.6 ± 4.4	22.4 ± 4.1	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Realtime ACT	31.2 ± 4.4	22.4 ± 4.1	40.6 ± 4.9	41.2 ± 4.8	22.4 ± 4.1	11.4 ± 3.0	60.2 ± 4.4	31.6 ± 4.4
Streaming DP	11.4 ± 3.0	41.6 ± 4.8	41.4 ± 4.6	22.4 ± 4.1	0.0 ± 0.0	0.0 ± 0.0	41.6 ± 4.8	30.4 ± 4.3
DreamVLA	31.2 ± 4.4	49.1 ± 9.1	43.5 ± 4.5	41.2 ± 4.8	40.6 ± 4.9	0.0 ± 0.0	49.4 ± 4.7	45.2 ± 4.8
Track-and-Intercept VLA	62.4 ± 3.5	51.2 ± 4.7	38.6 ± 1.6	71.8 ± 6.2	29.4 ± 0.3	34.8 ± 4.5	27.6 ± 2.2	33.6 ± 2.4
LaDi-WM (OpenVLA feat.)	48.2 ± 9.7	44.6 ± 1.8	52.4 ± 1.7	50.8 ± 3.8	41.2 ± 5.7	18.6 ± 3.7	36.4 ± 2.6	49.8 ± 1.8
AHEAD (ours)	$\mathbf{94.6 \pm 2.3}$	$\mathbf{87.4 \pm 3.2}$	$\mathbf{95.4 \pm 2.2}$	$\mathbf{95.8 \pm 2.0}$	$\mathbf{87.2 \pm 3.3}$	$\mathbf{79.4 \pm 3.8}$	$\mathbf{94.4 \pm 2.4}$	$\mathbf{95.6 \pm 2.1}$

Table 4: Physical robot experiments on a UFactory xArm 7. **Left**: five dynamic manipulation tasks (out of 30 attempts). **Right**: conveyor speed sensitivity (out of 10 attempts per speed point).

	Tasks					Speed sensitivity (cm/s)					
Method	**Static obj., mov. box**	**Static box, mov. obj.**	**Paddle hit**	**Stop ball**	**Catch proj.**	**0**	**5**	**10**	**15**	**20**	**25**
Open-loop VLA	0/30	0/30	0/30	0/30	0/30	10/10	0/10	0/10	0/10	0/10	0/10
Retargeting VLA	0/30	0/30	0/30	0/30	0/30	10/10	4/10	0/10	0/10	0/10	0/10
VLA + Fast Replan	0/30	0/30	0/30	0/30	0/30	–	–	–	–	–	–
Realtime ACT	9/30	5/30	9/30	2/30	0/30	10/10	8/10	0/10	0/10	0/10	0/10
Streaming Diffusion Policy	3/30	3/30	0/30	0/30	0/30	10/10	0/10	3/10	0/10	0/10	0/10
DreamVLA	12/30	9/30	1/30	12/30	0/30	10/10	2/10	0/10	0/10	0/10	0/10
AHEAD (ours)	**30/30**	**29/30**	**23/30**	**30/30**	**19/30**	**10/10**	**10/10**	**10/10**	**7/10**	**6/10**	**2/10**

limit guards halting the arm short of an otherwise geometrically-correct intercept, indicating the predicted target was reachable in principle but not from the current configuration. The remaining six split between the end-effector arriving just after the ball passed and the ball bouncing out of the net on contact. The speed-sensitivity sweep (right half of Table 4) shows the speed-robustness story transferring to hardware. AHEAD is the only method exceeding chance performance beyond 5 cm/s.

5.4 Ablation Studies

Table 5 reports mean success rate for each ablated configuration across the four scenarios in the relevant scene group (constant-velocity for motion estimator, spatial masking, and fixed horizon; acceleration/deceleration for velocity model). Per-scenario tables and the world model architecture ablation (samples S, encoder depth) are in Appendix E.

What each component buys. Explicit motion and the analytic kinematic update drive accuracy. Feeding the world model the same three frames' VLA tokens with no flow input and matched capacity reaches 74.2% against 93.5%, since sub-patch displacements are a noisy motion signal in pooled semantic tokens. Saliency masking and adaptive halting are accuracy-neutral on constant-velocity scenes, where full prediction and fixed $K{=}5$ match the default within error. Their contribution is fitting the latency budget and extending the method to chaotic motion. Predicting all 196 tokens raises latency to 250 ms, past the 200 ms budget (Table 14), and on Plinko at the 1600 ms window adaptive halting reaches 48.6% against 31.4% for fixed $K{=}5$ (Table 16), as post-collision variance truncates rollouts at the peg contact. Removing the alignment layer lowers Ballistic Catch to 61.1% (Table 19), still above every baseline in Table 3, so the 0.1M layer recovers fidelity rather than supplying the policy.

6 Conclusion

AHEAD augments a frozen VLA with a motion-aware latent world model so the robot acts on predicted future states rather than stale observations. A 4.9M-parameter wrapper around an unmodified

Table 5: Ablation summary. Mean success rate (%) $\pm$ standard error across four scenarios in the relevant scene group (constant-velocity for motion estimator, spatial masking, fixed horizon; acceleration/deceleration for velocity model). Full per-scenario tables in Appendix E.

Motion estimator		**Velocity model**		**Spatial masking**		**Fixed horizon K**	
RAFT (ours)	$\mathbf{93.5 \pm 1.8}$	Kinematic + accel (ours)	$\mathbf{90.7 \pm 1.1}$	Language + motion (ours)	$\mathbf{93.8 \pm 1.2}$	$K = 5$ (=adaptive)	$\mathbf{93.8 \pm 1.2}$
CoTracker3	84.0 ± 2.7	Kinematic only	82.5 ± 3.6	Full prediction	91.2 ± 2.7	$K = 8$	92.9 ± 1.1
Implicit 3-frame context	74.2 ± 3.1	Learned dynamics	78.7 ± 7.9	Motion only	84.0 ± 1.5	$K = 12$	86.3 ± 3.0
Farnebäck	68.9 ± 4.5	Accel-token only	78.5 ± 5.8	Language only	77.9 ± 3.9	$K = 3$	79.5 ± 1.8
Frame differencing	38.5 ± 3.4	Constant $V_k = V_0$	71.7 ± 10.4	Uniform random (30%)	46.4 ± 2.5	$K = 2$	60.2 ± 1.9
No velocity input	0.0 ± 0.0	Exponential decay	65.3 ± 11.0			$K = 1$	50.0 ± 4.0

7B OpenVLA backbone reaches 79 to 97% success across 20 dynamic simulation scenarios where the strongest baseline reaches 31 to 58%, and succeeds on $30/30$ and $29/30$ on two conveyor tasks, $30/30$ on a rolling-ball task, $23/30$ on paddle interception, and $19/30$ on projectile catching where every baseline fails entirely. The full pipeline runs in ≈ 158 ms per action step, within the ≈ 200 ms reactive manipulation budget. Ablations show that explicit RAFT-based motion estimation and the analytical kinematic update drive accuracy, while language-and-motion saliency and adaptive halting keep prediction within the latency budget. The broader contribution is an architectural template for adding predictive lookahead to pretrained VLAs without modifying or retraining the underlying model.

6.1 Limitations and Future Work

The constant-acceleration kinematic update breaks down under chaotic post-collision dynamics. Plinko-drop performance drops to 48.6% at the longest reaction window, illustrating the regime where the low-order kinematic assumption fails (Appendix D.1). Higher-order motion models or learned post-collision dynamics are natural extensions. RAFT-derived flow captures only image-plane motion, so scenes with significant depth motion would benefit from depth-augmented flow or multi-view inputs. The physical evaluation uses ≈ 200 in-lab xArm 7 trajectories, and transfer to substantially different platforms (bimanual systems, humanoids) is untested. The S trajectory samples capture dynamics-model uncertainty but share a single RAFT velocity estimate; sample variance can therefore be confidently wrong when flow is systematically wrong, and folding RAFT confidence into the velocity noise scale, together with epistemic ensembles, would improve calibration. Specular and textureless surfaces, where flow is unreliable, and wrist-mounted cameras, whose ego-flow would need to be removed via forward kinematics, are untested. Predicted targets never bypass the workspace and joint-limit guards, but arm speed is not yet gated by prediction variance. AHEAD also treats the underlying VLA as fixed; a faster VLA backbone would directly benefit the predict-then-act wrapper, and we leave the joint optimization of backbone and world model to future work.

References

[1] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.

[2] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, J. Luo, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine. Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems*, Delft, Netherlands, 2024.

[3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, S. Levine, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of The 7th Conference on Robot Learning*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183, 2023.

[4] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky. π_0: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.

[5] D. M. Wolpert and M. Kawato. Multiple paired forward and inverse models for motor control. *Neural Networks*, 11(7–8):1317–1329, 1998.

[6] R. Shadmehr and F. A. Mussa-Ivaldi. Adaptive representation of dynamics during learning of a motor task. *Journal of Neuroscience*, 14(5):3208–3224, 1994.

[7] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.

[8] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Proceedings of Robotics: Science and Systems (RSS)*, 2023.

[9] J. Zhang, Y. Guo, X. Chen, Y.-J. Wang, Y. Hu, C. Shi, and J. Chen. HiRT: Enhancing robotic control with hierarchical robot transformers. In *Conference on Robot Learning (CoRL)*, 2024.

[10] Q. Bu, H. Li, L. Chen, J. Cai, J. Zeng, H. Cui, M. Yao, and Y. Qiao. Towards synergistic, generalized, and efficient dual-system for robotic manipulation. *arXiv preprint arXiv:2410.08001*, 2024.

[11] H. Xie, B. Wen, J. Zheng, Z. Chen, F. Hong, H. Diao, and Z. Liu. DynamicVLA: A vision-language-action model for dynamic object manipulation. *arXiv preprint arXiv:2601.22153*, 2026.

[12] J. Tang, Y. Sun, Y. Zhao, S. Yang, Y. Lin, Z. Zhang, J. Hou, Y. Lu, Z. Liu, and S. Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.

[13] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[14] J. Bruce, M. Dennis, A. Edwards, J. Parker-Holder, Y. Shi, E. Hughes, M. Lai, A. Mavalankar, R. Steiber, C. Apps, Y. Aytar, S. Bechtle, F. Behbahani, S. Chan, N. Heess, L. Gonzalez, S. Osindero, S. Ozair, S. Reed, J. Zhang, K. Zolna, J. Clune, N. de Freitas, S. Singh, and T. Rocktäschel. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.

[15] J. Cen, C. Yu, H. Yuan, Y. Jiang, S. Huang, J. Guo, X. Li, Y. Song, H. Luo, F. Wang, D. Zhao, and H. Chen. WorldVLA: Towards autoregressive action world model. *arXiv preprint arXiv:2506.21539*, 2025.

[16] Y. Huang, J. Zhang, S. Zou, X. Liu, R. Hu, and K. Xu. LaDi-WM: A latent diffusion-based world model for predictive manipulation. In *Conference on Robot Learning (CoRL)*, 2025.

[17] J. Bjorck et al. GR00T N1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025. NVIDIA.

[18] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint arXiv:1912.01603*, 2020.

[19] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering Atari with discrete world models. In *International Conference on Learning Representations (ICLR)*, 2021.

[20] N. Hansen, H. Su, and X. Wang. TD-MPC2: Scalable, robust world models for continuous control. *arXiv preprint arXiv:2310.16828*, 2024.

[21] M. Yang, Y. Du, K. Ghasemipour, J. Tompson, D. Schuurmans, and P. Abbeel. Learning interactive real-world simulators. *arXiv preprint arXiv:2310.06114*, 2024.

[22] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou. Information theoretic MPC for model-based reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 1714–1721, 2017.

[23] A. Nagabandi, G. Keet, R. S. Fearing, and S. Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 7559–7566, 2018.

[24] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. In *arXiv preprint arXiv:1812.00568*, 2018.

[25] J. B. Rawlings, D. Q. Mayne, and M. M. Diehl. Model predictive control: Theory, computation, and design. *Nob Hill Publishing*, 2017.

[26] D. Morrison, P. Corke, and J. Leitner. Closing the loop for robotic grasping: A real-time, generative grasp synthesis approach. In *Robotics: Science and Systems*, 2018.

[27] M. Kopicki, D. Belter, and J. Wyatt. Dynamic grasp and trajectory planning for moving objects. *Autonomous Robots*, 43(1):175–189, 2019.

[28] Y. Guo, L. X. Shi, J. Chen, and C. Finn. Ctrl-World: A controllable generative world model for robot manipulation. *arXiv preprint arXiv:2510.10125*, 2025.

[29] Y. Guo, T. Lee, L. X. Shi, J. Chen, P. Liang, and C. Finn. VLAW: Iterative co-improvement of vision-language-action policy and world model. *arXiv preprint arXiv:2602.12063*, 2026.

[30] C.-L. Cheang et al. GR-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2407.14615*, 2024.

[31] A. Bardes, Q. Garrido, J. Ponce, X. Chen, M. Rabbat, Y. LeCun, M. Assran, and N. Ballas. V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv preprint arXiv:2506.09985*, 2025.

[32] S. Nair, A. Rajeswaran, V. Kumar, C. Finn, and A. Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning (CoRL)*, 2023.

[33] K. Grauman, A. Westbury, E. Byrne, Z. Chavis, A. Furnari, R. Girdhar, J. Hamburger, H. Jiang, M. Liu, X. Liu, M. Martin, T. Nagarajan, I. Radosavovic, S. K. Ramakrishnan, F. Ryan, J. Sharma, M. Wray, M. Xu, E. Z. Xu, C. Zhao, S. Bansal, D. Batra, V. Cartillier, S. Crane, T. Do, M. Doulaty, A. Erapalli, C. Feichtenhofer, A. Fragomeni, Q. Fu, C. Fuegen, A. Gebreselasie, C. Gonzalez, J. Hillis, X. Huang, Y. Huang, W. Jia, W. Khoo, J. Kolar, S. Kottur, A. Kumar, F. Landini, C. Li, Y. Li, Z. Li, K. Mangalam, R. Modhugu, J. Munro, T. Murrell, T. Nishiyasu, W. Price, P. R. Puentes, M. Ramazanova, L. Sari, K. Somasundaram, A. Southerland, Y. Sugano, R. Tao, M. Tran, V. Veeriah, Q. Wan, Y. Wang, M. Wu, Y. Xie, M. Yan, E. Z. Xu, Y. Zhao, V. Baiyya, B. Ghanem, J. Shi, M. Smith, L. Torresani, S. Kong, and D. Parikh. Ego4D: Around the world in 3,000 hours of egocentric video. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 18995–19012, 2022.

[34] D. Damen, H. Doughty, G. M. Farinella, S. Fidler, A. Furnari, E. Kazakos, D. Moltisanti, J. Munro, T. Perrett, W. Price, and M. Wray. Rescaling egocentric vision: Collection, pipeline and challenges for EPIC-KITCHENS-100. *International Journal of Computer Vision*, 130(1): 33–55, 2022.

[35] R. Goyal, S. E. Kahou, V. Michalski, J. Materzynska, S. Westphal, H. Kim, V. Haenel, I. Fruend, P. Yianilos, M. Mueller-Freitag, F. Hoppe, C. Thurau, I. Bax, and R. Memisevic. The "Something Something" video database for learning and evaluating visual common sense. In *IEEE International Conference on Computer Vision (ICCV)*, pages 5843–5851, 2017.

[36] A. Khazatsky, K. Pertsch, S. Nair, A. Balakrishna, S. Dasari, S. Karamcheti, S. Nasiriany, M. K. Srirama, L. Y. Chen, K. Ellis, P. D. Fagan, J. Hejna, M. Itkina, M. Lepert, Y. J. Ma, P. T. Miller, J. Wu, S. Belkhale, S. Dass, H. Ha, A. Jain, A. Lee, Y. Lee, M. Memmel, S. Park, I. Radosavovic, K. Wang, A. Zhan, K. Black, C. Chi, K. B. Hatch, S. Lin, J. Lu, J. Mercat, A. Rehman, P. R. Sanketi, A. Sharma, C. Devin, B. Ichter, Q. Vuong, H. Walke, B. Wulfe, T. Xiao, J. H. Yang, A. Yousefi, M. R. Zoghi, D. Sadigh, L. Pinto, C. Finn, and S. Levine. DROID: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.

[37] H. Walke, K. Black, A. Lee, M. J. Kim, M. Du, C. Zheng, T. Zhao, P. Hansen-Estruch, Q. Vuong, A. He, V. Myers, K. Fang, C. Finn, and S. Levine. BridgeV2: Manipulator control pre-training across tasks and environments. In *Conference on Robot Learning (CoRL)*, 2023.

[38] Z. Teed and J. Deng. RAFT: Recurrent all-pairs field transforms for optical flow. In *European Conference on Computer Vision (ECCV)*, pages 402–419, 2020.

[39] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le. Flow matching for generative modeling. In *International Conference on Learning Representations (ICLR)*, 2023.

[40] P. Esser, S. Kulal, A. Blattmann, R. Entezari, J. Müller, H. Saini, Y. Levi, D. Lorenz, A. Sauer, F. Boesel, D. Podell, T. Dockhorn, Z. English, K. Lacey, A. Goodwin, Y. Marek, and R. Rombach. Scaling rectified flow transformers for high-resolution image synthesis. In *International Conference on Machine Learning (ICML)*, 2024.

[41] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 5026–5033, 2012.

[42] S. Haddadin, S. Parusel, L. Johannsmeier, S. Golz, S. Gabl, F. Walch, M. Sagsetter, T. Kollar, and A. Albu-Schäffer. The Franka Emika robot: A reference platform for robotics research and education. *IEEE Robotics & Automation Magazine*, 29(1):46–64, 2022.

[43] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.

[44] W. Zhang, H. Liu, Z. Qi, Y. Wang, X. Yu, J. Zhang, R. Dong, J. He, H. Wang, Z. Zhang, L. Yi, W. Zeng, and X. Jin. DreamVLA: A vision-language-action model dreamed with comprehensive world knowledge. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[45] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 44(10–11):1684–1704, 2024.

[46] A. Dekhne, G. Hastings, J. Murnane, and F. Schür. Automation in logistics: Big opportunity, bigger uncertainty. *McKinsey & Company Report*, 2019.

[47] N. Correll, K. E. Bekris, D. Berenson, O. Brock, A. Causo, K. Hauser, K. Okada, A. Rodriguez, J. M. Romano, and P. R. Wurman. Analysis and observations from the first Amazon picking challenge. In *IEEE Transactions on Automation Science and Engineering*, volume 15, pages 172–188, 2018.

[48] S. N. Syed, Y. Ahuja, A. Jakobsson, and J. Ichnowski. ExpReS-VLA: Specializing vision-language-action models through experience replay and retrieval. *arXiv preprint arXiv:2511.06202*, 2025.

[49] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022.

[50] C. Finn and S. Levine. Deep visual foresight for planning robot motion. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 2786–2793, 2017.

[51] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg. DayDreamer: World models for physical robot learning. *arXiv preprint arXiv:2206.14176*, 2023.

[52] B. Huang, J. Yu, and S. Jain. EARL: Eye-on-hand reinforcement learner for dynamic grasping with active pose estimation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2023. arXiv:2310.06751.

[53] K. Yamamoto, H. Ito, H. Ichiwara, H. Mori, and T. Ogata. Real-time motion generation and data augmentation for grasping moving objects with dynamic speed and position changes. *arXiv preprint arXiv:2309.12547*, 2023.

[54] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[55] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using RNN encoder-decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734, 2014.

[56] Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning (ICML)*, pages 1050–1059, 2016.

[57] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. *arXiv preprint arXiv:2303.05499*, 2024.

[58] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard. CALVIN: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks. In *IEEE Robotics and Automation Letters*, volume 7, pages 7327–7334, 2022.

[59] S. James, Z. Ma, D. R. Arrojo, and A. J. Davison. RLBench: The robot learning benchmark & learning environment. *IEEE Robotics and Automation Letters*, 5(2):3019–3026, 2020.

[60] Open X-Embodiment Collaboration. Open X-embodiment: Robotic learning datasets and RT-X models. *arXiv preprint arXiv:2310.08864*, 2024.

[61] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

[62] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[63] D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[64] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[65] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer. Sigmoid loss for language image pre-training. *arXiv preprint arXiv:2303.15343*, 2023.

[66] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning (ICML)*, pages 8748–8763, 2021.

[67] E. A. Theodorou, J. Buchli, and S. Schaal. A generalized path integral control approach to reinforcement learning. In *Journal of Machine Learning Research*, volume 11, pages 3137–3181, 2010.

[68] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30, 2017.

[69] B. Lakshminarayanan, A. Pritzel, and C. Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[70] R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1):79–87, 1999.

[71] S. Abu-El-Haija, N. Kothari, J. Lee, P. Natsev, G. Toderici, B. Varadarajan, and S. Vijayanarasimhan. YouTube-8M: A large-scale video classification benchmark. In *arXiv preprint arXiv:1609.08675*, 2016.

[72] J. L. Ba, J. R. Kiros, and G. E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[73] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollár, and R. Girshick. Segment anything. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, 2023.

A Full Architecture Pipeline

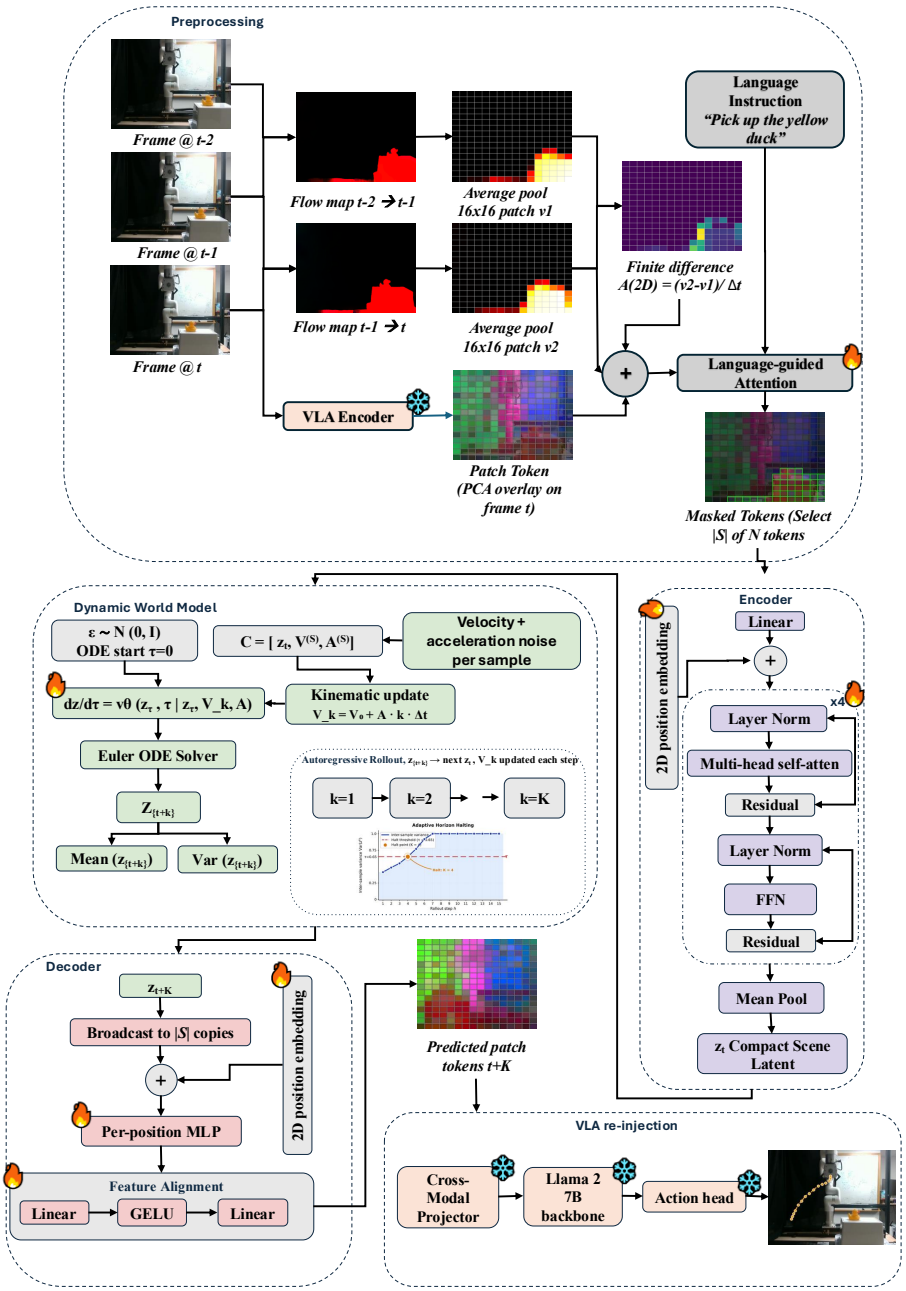


Figure 3: **AHEAD full architecture.** The preprocessing stage computes optical flow between three consecutive frames using RAFT [38], pools to per-patch velocity, and finite-differences to recover per-patch acceleration. A frozen OpenVLA vision encoder produces N patch tokens, which are gated by language-guided cross-attention to select a task-relevant subset $\mathcal{S}$ of $|\mathcal{S}|$ tokens (typically 30 to 60 for N=196). A 4-layer transformer encoder compresses the selected motion-enriched tokens into a compact scene latent z_t. The dynamic world model rolls forward via conditional flow matching with analytical kinematic conditioning $V_k = V_0 + A \cdot k \cdot \Delta t$, drawing S=5 samples per step. The rollout halts when inter-sample variance exceeds threshold τ_u, or at $K_{\max}$ steps if the threshold is never crossed. The decoder reconstructs predicted patch tokens $\hat{v}^{(\mathcal{S})}_{t+K}$, which are spliced into the full N-token grid (unselected positions pass through unchanged) and injected into the frozen OpenVLA backbone to produce the action $a_t = \pi(\hat{v}_{t+K}, e_\ell)$. Components trained from scratch (∼4.9M parameters total) are marked with a flame icon; components inherited frozen from OpenVLA are marked with a snowflake icon.

B Method Details

This appendix expands on Section 4. Section B.1 gives encoder, decoder, and dynamics-model dimensions. Section B.2 compares conditional flow matching against diffusion and dropout-based GRU dynamics. Section B.3 reports the per-component latency breakdown. Section B.4 details the language-and-motion saliency module. Section B.5 details RAFT-based motion estimation. Section B.6 derives the constant-acceleration kinematic update. Section B.7 describes feature-alignment-layer training. Section B.8 gives the full training protocol.

B.1 Architecture Details

AHEAD adds $\approx$ 4.9M trainable parameters on top of the $\approx$ 7B frozen OpenVLA [1] backbone, distributed across components as follows: world model encoder $\approx$ 2.6M, flow matching dynamics $\approx$ 1.1M, decoder $\approx$ 0.8M, language-and-motion saliency module $\approx$ 0.3M, and per-token feature alignment layer $\approx$ 0.1M. The auxiliary modules thus constitute under 0.1% of the total parameter count, so the predict-then-act loop is a lightweight wrapper around an unmodified OpenVLA.

Encoder. A 4-layer transformer with 256-dim hidden state compresses the selected motion-enriched tokens into a compact latent. Each selected token receives a learnable 2D positional embedding indexed by its (row, col) patch position, so spatial positions are consistent with the frozen VLA's representation,

$$z_t = g_{\text{enc}}\big(\tilde{v}_t^{(\mathcal{S})}\big) \in \mathbb{R}^{d_z}, \tag{5}$$

where $\tilde{v}_t^{(\mathcal{S})}$ denotes the motion-enriched tokens indexed by $\mathcal{S}$, and d_z is the latent dimension.

Decoder. An MLP decoder reconstructs VLA-compatible patch tokens from the predicted latent,

$$\hat{v}_{t+k}^{(\mathcal{S})} = g_{\text{dec}}(z_{t+k}) \in \mathbb{R}^{|\mathcal{S}| \times d}. \tag{6}$$

The decoder splices aligned predicted tokens into the full N-token grid, keeping unmasked tokens from the current observation,

$$\hat{v}_{t+K}[i] = \begin{cases} g_{\text{align}}\big(\hat{v}_{t+K}^{(\mathcal{S})}[i]\big) & \text{if } i \in \mathcal{S}, \\ v_t[i] & \text{otherwise.} \end{cases} \tag{7}$$

Static background has not changed between t and $t{+}K$, so the current observation is the correct prediction for those tokens.

What the spliced grid represents. The splice is a functional surrogate for the future observation, not a visual reconstruction. Predicted tokens are never relocated across the grid. A ballistic ball traverses 8 to 15 patches over the rollout, and the selected tokens encode its future kinematic state at their original positions. The alignment layer (Section B.7) is trained so that the spliced grid drives the frozen decoder to the action it would produce on the true future frame. Table 19 shows that the alignment layer recovers fidelity rather than supplying the policy.

B.2 Alternative Architectures Considered

The world model must produce high-quality predictions and complete within the per-step latency budget. We considered three candidate architectures.

Diffusion-based world models [61] yield high-quality samples but require 100 or more denoising steps, placing each rollout at $\approx$ 380 ms, outside the budget.

Deterministic GRUs [55] run in $\approx$ 45 ms but produce a single-point prediction with no uncertainty estimate. Obtaining one via Monte Carlo Dropout [56] requires 10 forward passes at $10 \times 45 = 450$ ms, also outside the budget.

Conditional flow matching learns a continuous normalizing flow from noise to the predicted future state distribution, requiring 5 integration steps for high-quality samples ($\approx$ 50 ms for $S{=}5$ samples).

Multiple samples yield both a mean prediction and a natural uncertainty estimate from trajectory sample variance, at no additional overhead.

AHEAD adopts conditional flow matching as the dynamics model on this basis.

B.3 Latency Breakdown

Table 6 reports the per-component latency breakdown on H100. The $\Delta t_{\max} \approx 200$ ms budget is consistent with human visuomotor reaction loops [5] and the operating regime of prior reactive manipulation policies [26, 7].

Table 6: End-to-end latency for one action step on an NVIDIA H100 80GB GPU at 224×224 input resolution in bfloat16 precision. AHEAD uses OpenVLA [1] with OpenVLA-OFT-style parallel decoding and action chunking for the frozen action head. The full pipeline completes in ≈ 158 ms, within the $\Delta t_{\max} \approx 200$ ms budget required for reactive manipulation. Per-component values include framework overhead from current PyTorch eager-mode execution and the Python rollout loop; this overhead could be further reduced with compilation (e.g., `torch.compile`, CUDA graphs).

Component	**Latency**
RAFT optical flow [38] ($\times 2$)	≈ 20 ms
Motion pooling and concatenation	≈ 2 ms
World model ($S = 5$ flow matching samples)	≈ 40 ms
Decoder and feature alignment	≈ 6 ms
VLA forward pass (parallel decoding)	≈ 70 ms
Framework and I/O overhead	≈ 20 ms
Total end-to-end	$\mathbf{\approx 158}$ **ms**

The ≈ 158 ms figure is the replan interval, not the control rate. Action chunks execute at 30 Hz, the wrapper adds ≈ 68 ms over the ≈ 90 ms reactive loop, and the 3 to 5 step lookahead spans that delay. Physical experiments run on a single NVIDIA RTX 4090 at 171 ms per action step.

B.4 Language-and-Motion Saliency Details

The frozen VLA's language encoder produces embeddings $e_\ell \in \mathbb{R}^{L \times d_l}$ for instruction ℓ. A learned cross-attention layer attends from the N token positions of $\tilde{v}_t$ to e_ℓ, producing per-token relevance scores that a sigmoid activation normalizes to $[0, 1]$,

$$\widetilde{M} = \sigma\big(\mathrm{CrossAttn}(\tilde{v}_t,\ e_\ell)\big) \in [0,1]^N. \tag{8}$$

A 1-token morphological dilation expands $\widetilde{M}$ to capture object boundaries.

The motion-saliency floor uses $\alpha_{\text{motion}} = 0.5$ in Equation (1). The max operator establishes this floor for motion-salient tokens, so that fast-moving objects pass through even when the instruction does not ground there; tokens with high language relevance are unaffected, since their $\widetilde{M}_i$ already dominates. We calibrate τ_{flow} on a held-out set of static lab scenes by measuring the 99th-percentile of per-token velocity magnitude under camera noise alone.

Section 5 ablates the language and motion-saliency signals against uniform masking to characterize the compute-accuracy tradeoff. The saliency layer needs no mask labels and trains through the world-model objective.

Robot-arm patches. The motion floor in Equation (1) also fires on the robot's own moving arm. Arm motion is robot-induced rather than independent, and the world model has no action conditioning with which to predict it. We identify arm patches by projecting the robot's link geometry into the image through forward kinematics and the calibrated camera extrinsics, and mark every patch the projection overlaps. Excluding these patches from $\mathcal{S}$ raises Ballistic Catch from 87.4% to 92.2% (Table 19). All other results in this paper keep arm patches in $\mathcal{S}$, and we recommend the exclusion as the default configuration going forward.

B.5 Motion Estimation Details

To condition on per-token kinematic state rather than asking the world model to infer motion from visual features alone, AHEAD estimates both velocity and acceleration from three consecutive observations using RAFT [38] and finite differencing.

Velocity. RAFT computes a dense optical flow field $F_{t-1:t} \in \mathbb{R}^{H_o \times W_o \times 2}$ between o_{t-1} and o_t. Patch-level average-pooling over each 16×16 region yields a per-token velocity,

$$V_i = \mathrm{AvgPool}_i\big(F_{t-1:t}\big) \in \mathbb{R}^2, \quad i = 1, \ldots, N. \tag{9}$$

Average-pooling suffices because task-relevant objects span multiple patches. Interior patches receive clean velocity signals, and the 2 to 4 boundary patches with mixed signals contribute negligibly to prediction error.

Acceleration. A second flow field $F_{t-2:t-1}$ between o_{t-2} and o_{t-1} is pooled identically to produce a previous velocity field V^{prev}. Finite-differencing the two velocity fields yields per-token acceleration,

$$A_i = \frac{V_i - V_i^{\text{prev}}}{\Delta t} \in \mathbb{R}^2, \quad i = 1, \ldots, N, \tag{10}$$

where Δt is the inter-frame interval. The pipeline concatenates each token's velocity and acceleration into a motion descriptor $[V_i;\, A_i] \in \mathbb{R}^4$ and appends it to the patch token to form the motion-enriched representation $\tilde{v}_t \in \mathbb{R}^{N \times (d+4)}$.

Why per-token and why both signals. Per-token motion descriptors preserve spatial structure, so the world model can reason that a specific token contains a moving object with specific velocity and acceleration, where a global motion summary would lose the spatial binding between motion and object identity. Velocity alone suffices for constant-motion scenarios such as conveyors and linear pushes, but acceleration is essential for trajectories that speed up or slow down, including objects accelerating under gravity, decelerating against friction, or undergoing post-contact changes. Section 5 ablates velocity-only, acceleration-only, and combined motion conditioning.

B.6 Kinematic Update Derivation

At step $k{=}1$, real RAFT-derived velocity and acceleration condition the model. At steps $k \geq 2$, no new observation is available, so AHEAD extrapolates the velocity conditioning analytically under constant-acceleration kinematics per Equation (3). This propagation captures the dominant trajectory curvature in acceleration-driven scenarios such as gravitational pull and friction decay, without requiring the world model to learn full physics from data.

To capture motion-estimation uncertainty, AHEAD injects small Gaussian noise $\sigma = 0.05 \cdot \|V_i\|$ into the velocity conditioning independently per sample, causing trajectories to diverge when predictions are sensitive to exact velocity values and to agree when they are robust. The resulting sample variance is predominantly aleatoric spread plus velocity-noise sensitivity, and it correlates with realized prediction error at $r = 0.78$. The conditioning V_k never depends on model output (Equation (3)), training rolls out $K_{\text{train}} = 8$ steps under the model's own predictions, and halting cuts the rollout before error compounds.

B.7 Feature Alignment Layer

MSE reconstruction treats all feature dimensions equally, but the VLA action decoder is sensitive to some dimensions, such as spatial relationships critical for grasping, and insensitive to others, such as background texture. To correct this systematic distortion, AHEAD adds a per-token feature alignment layer that maps decoded features to the action decoder's preferred manifold.

We train the alignment layer separately from the dynamics model on paired real and decoded features, directly optimizing action error,

$$\mathcal{L}_{\text{align}} = \big\|\pi(g_{\text{align}}(\hat{v}_{t+k}), e_\ell) - \pi(v_{t+k}, e_\ell)\big\|^2. \tag{11}$$

For training purposes, we compute the loss using full-grid action predictions: the predicted tokens $g_{\text{align}}(\hat{v}_{t+k}^{(S)})$ are spliced into the current observation grid per Eq. (7) before being passed to π.

The alignment layer is a two-layer MLP with GELU activation and a residual connection, applied per-token. Training uses paired samples $(v_{t+k}, \hat{v}_{t+k})$ drawn from the validation portion of the pretraining corpus, with v_{t+k} encoded from the actual future frame and $\hat{v}_{t+k}$ predicted by the trained world model. The frozen action decoder π provides supervision through the action error gradient. Separating alignment training from world-model training avoids the optimization conflict between reconstruction quality (MSE on features) and action accuracy (downstream policy loss).

B.8 Training Protocol

Why manipulation video pretraining. Collecting robot trajectories is expensive, which limits the scale of training data for robot-specific world models. The world model operates in VLA feature space, which the frozen encoder ϕ has already trained on diverse internet-scale data. Any video containing object motion in manipulation-relevant contexts therefore provides useful training signal, regardless of whether a robot appears in the scene. This motivates a curated pretraining corpus focused on tabletop interactions and hand-object dynamics, rather than broad egocentric video datasets dominated by outdoor activities and whole-body motion that lie far from the deployment distribution.

Phase 1: video encoding (one-time). For each sequence in the training corpus, the frozen encoder ϕ encodes every frame into patch tokens, and RAFT computes inter-frame flow,

$$v_i = \phi(o_i), \qquad V_i = \text{AvgPool}\big(\text{RAFT}(o_i, o_{i+1})\big), \quad i = 1, \ldots, N_{\text{frames}}-1. \tag{12}$$

For egocentric sources (EPIC-Kitchens [34] and Something-Something V2 [35]), we estimate and subtract the dominant homography from each flow field before pooling, so that residual flow captures independent object motion rather than camera ego-motion. Table 7 summarizes the training data sources.

Table 7: Training data sources for world model pretraining and fine-tuning. The deployment setup uses Franka in simulation and a UFactory xArm 7 in the physical lab. Pretraining uses six public manipulation video sources spanning egocentric human activity and robot demonstrations from multiple embodiments.

Dataset	**Content**	**Sequences**	**Usage**
EPIC-Kitchens [34]	First-person kitchen manipulation	$\approx$ 50K	Pretrain
Something-Something V2 [35]	Hand-object interactions (174 actions)	$\approx$ 220K	Pretrain
DROID [36]	Diverse robot manipulation (86 tasks)	$\approx$ 76K	Pretrain
Bridge V2 [37]	WidowX tabletop manipulation	$\approx$ 60K	Pretrain
CALVIN [58]	Simulated Franka manipulation	$\approx$ 100K	Pretrain
OXE [60] (xArm subset)	xArm 5/6/7 from OXE	$\approx$ 150K	Pretrain
xArm 7 (in-lab)	Physical deployment	$\approx$ 200	Real fine-tuning
Total		$\approx$ 656K	

Phase 2: curriculum pretraining. Pretraining follows a three-stage curriculum to build motion understanding progressively. Stage 1 uses simple linear motion from Something-Something V2 and Bridge V2 [37]. Stage 2 adds rotational and multi-object motion from EPIC-Kitchens and DROID [36]. Stage 3 mixes all sources with domain-similarity weighting. Each sequence receives a sampling weight that is the product of two factors: a feature-distance factor (1 to 5 times based on Fréchet distance between the sequence's VLA features and the deployment distributions of Franka in simulation and xArm 7 in the physical lab) and a source-type factor (1 for egocentric sources, 3 for fixed-camera robot and simulation sources). The latter accounts for ego-motion absence and closer match to the deployment setting in fixed-camera sources (CALVIN, DROID, Bridge V2, and the OXE xArm subset). We uniformly sample $\approx$ 150K trajectories from the Open-X Embodiment [60] xArm subset to balance its contribution against the egocentric and other robot sources.

We jointly train the encoder, dynamics model, and decoder. We train the dynamics model with the standard conditional flow matching objective. We train the encoder and decoder with an MSE reconstruction loss over the rollout horizon K_{train},

$$\mathcal{L}_{\text{pre}} = \frac{1}{K_{\text{train}}} \sum_{k=1}^{K_{\text{train}}} \left\| g_{\text{dec}}(z_{t+k}) - v_{t+k} \right\|^2, \quad z_{t+k} \sim p\Big(\cdot \mid z_{t+k-1}, V_k^{(\mathcal{S})}, A^{(\mathcal{S})} \Big), \tag{13}$$

where z_{t+k} is sampled autoregressively from the dynamics distribution defined in Equation (2), and V_k updates analytically per Equation (3). MSE on features suffices during pretraining because if $\hat{v}_{t+k} \approx v_{t+k}$ in feature space, then the action error $\|\pi(\hat{v}_{t+k}, e_\ell) - \pi(v_{t+k}, e_\ell)\|$ is also small. We train the feature alignment layer separately afterward on paired real and decoded features per Equation (11).

Sensitivity to corpus size. No evaluation scene or object appears in the pretraining corpus. Retraining the world model on a random 25% of the corpus lowers success by 2.9 points averaged over the four constant-velocity scenes.

Phase 3: fine-tuning on robot data. A short fine-tuning phase on ≈ 200 xArm 7 sequences collected in our lab closes the remaining domain gap between OXE xArm trajectories and the specific camera placement, lighting, and gripper of the deployment setup. Fine-tuning uses the same MSE loss for 5 epochs.

Hyperparameters. Table 8 lists the key training hyperparameters.

Table 8: World model training hyperparameters.

Hyperparameter	**Value**
Latent dimension d_z	256
Encoder layers / hidden dim	4 / 256
Encoder attention heads	8
Flow matching ODE steps	5 (Euler)
Trajectory samples S	5
Maximum horizon $K_{\max}$	10
Velocity noise σ	$0.05 \times \|V_i\|$
Uncertainty threshold τ_u	90th-percentile (training)
Optimizer	AdamW (β_1=0.9, β_2=0.999)
Learning rate (peak)	3×10^{-4}
Learning rate schedule	Cosine decay, 2K warmup steps
Weight decay	1×10^{-2}
Batch size (pretraining)	256
Batch size (fine-tuning)	64
Sequence length (frames)	16
Training horizon K_{train}	8
Pretraining steps	200K
Fine-tuning epochs	5
Gradient clipping (L2 norm)	1.0
Mixed precision	bfloat16
Hardware	8×NVIDIA H100 80GB

C Experimental Setup

This appendix expands on the experimental setup summarized in Section 5.1. Section C.1 details the simulation environments. Section C.2 details the physical xArm 7 setup. Section C.3 gives baseline implementation details. Section C.4 specifies per-scenario success criteria. Section C.5 details the statistical protocol. Section C.6 reports compute resources.

C.1 Simulation Environments

All simulation experiments use MuJoCo [41] with a Franka Emika Panda [42] arm, a parallel-jaw gripper, and a fixed third-person RGB camera rendering at 224×224 resolution. Control runs at 30 Hz. Scenes are procedurally generated with randomized object positions, motion parameters, lighting direction, and camera-relative pose.

Constant-velocity scenarios. Four scenarios test linear independent motion.

Conveyor + cup. A conveyor belt transports a red cup across the workspace at a randomized speed between 5 and 25 cm/s. The robot must pick the cup before it leaves the workspace. Belt speed, cup spawn position along the belt, and belt height are randomized per episode.

Beam + cup. A red cup slides along a horizontal beam at a randomized speed between 5 and 20 cm/s. The robot must grasp the cup from above. Beam orientation and starting position are randomized.

Pole push + cup. A pole pushes a cup laterally across the workspace at a randomized speed between 8 and 22 cm/s. The robot must intercept the cup. Push direction and starting offset are randomized.

Rolling ball. A ball rolls across the table at a constant horizontal velocity between 10 and 30 cm/s. The robot must stop the ball.

Acceleration / deceleration scenarios. The same four scene topologies, replacing constant velocity with acceleration profiles. Conveyor and beam scenarios add either a linearly increasing or decreasing belt speed over the episode. Pole push adds frictional deceleration. Rolling ball adds gravitational acceleration on an inclined surface (slope randomized between 5 and 15 degrees).

Complex dynamic scenarios. Eight scenarios test capabilities beyond single-object linear motion.

Air Hockey (AH). A puck moves on an air-hockey-style frictionless surface at a randomized initial velocity. The robot, holding a paddle, must deflect the puck.

Ballistic Catch (BC). A ball is launched on a parabolic trajectory toward the robot's workspace. The robot must catch the ball before it reaches the floor.

Multi-Object (MO). Multiple objects (3 to 5) move simultaneously on linear trajectories. The robot must pick the language-specified object.

Place in Box (PiB). A conveyor-mounted box passes through the workspace at 10 to 20 cm/s. The robot must place a stationary object into the moving box.

Multi-Balls (MB). Three balls launched into the robot workspace. The robot must intercept the language-specified ball.

Occlusion (Occ). A launched ball passes behind a vertical occluder for a randomized interval (300 to 600 ms) before reappearing. The robot must catch the object after it emerges.

Mid-flight trajectory change (Mid). A launched ball on a parabolic trajectory deflects mid-flight from an unexpected contact, abruptly changing direction. The robot must adjust its intercept target.

Language + Multi-Object (L+MO). Three to five moving objects and two boxes of similar appearance but different language descriptions (e.g., "blue box," "brown box"). The robot must pick the instruction-specified object and bin it in the instruction-specified box.

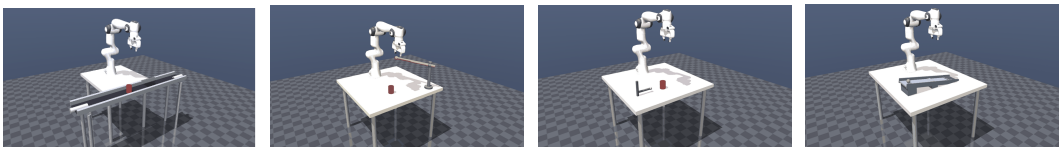
Stress-test scenarios. Three scenarios deliberately exceed the low-order kinematic assumption from Section 3; results are in Appendix D.1.

Multiple deflection. A ball bounces off two intermediate surfaces before reaching the workspace.

Occlusion deflection. A combination of occlusion and a single deflection during the occluded interval.

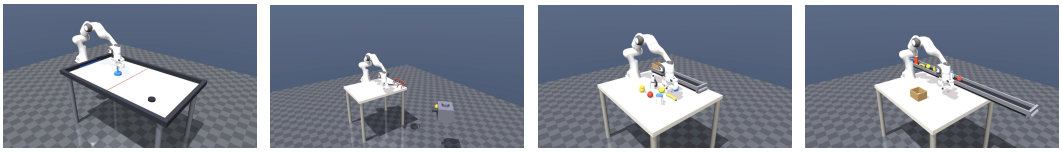
Plinko drop. A ball falls through a peg array, with each peg contact randomizing its trajectory; the robot must catch the ball at the bottom.

Simulation scene visualizations. Figure 4 shows the four constant-velocity scenarios. Figure 5 shows the eight complex scenarios. Figure 6 shows the three stress-test scenarios.

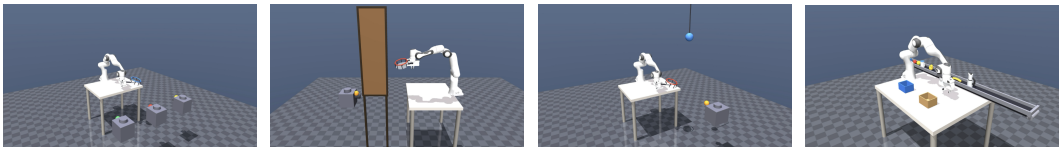


(a) Conveyor + cup (b) Beam + cup (c) Pole push + cup (d) Rolling ball

Figure 4: Constant-velocity simulation scenarios. The cup or ball moves at a fixed velocity through the workspace; the robot must intercept and grasp.

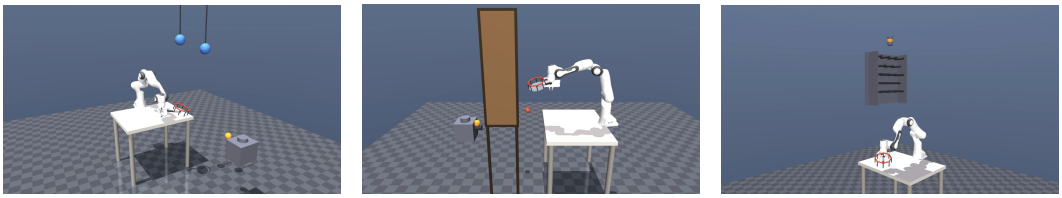


(a) Air Hockey (AH) (b) Ballistic Catch (BC) (c) Place in Box (PiB) (d) Multi-Object (MO)



(e) Multi-Balls (MB) (f) Occlusion (Occ) (g) Mid-flight Δ (Mid) (h) Lang. + Multi-obj. (L+MO)

Figure 5: Complex dynamic simulation scenarios. The eight scenes test reactive contact, ballistic trajectories, multi-object selection, occlusion, mid-flight deflection, and language-conditioned target selection.



(a) Multiple deflection (b) Occlusion deflection (c) Plinko drop

Figure 6: Stress-test simulation scenarios that exceed the low-order kinematic motion assumption from Section 3. Each ball undergoes multiple chaotic deflections; AHEAD degrades gracefully but does not match its in-distribution success rates.

C.2 Physical Robot Setup

Robot. A UFactory xArm 7 (7-DOF) with a parallel-jaw gripper. The robot operates within a safety bounding box of approximately $1.2 \times 0.8 \times 0.6$ m centered in front of the base, with joint-limit guards that halt motion if any joint approaches the manufacturer limits or the workspace bound.

Camera. A single Intel RealSense D435 mounted on a fixed third-person tripod approximately 1.2 m from the robot base. Only the RGB stream is used; depth is not consumed by AHEAD or any baseline. Images are downsampled to 224×224 to match the OpenVLA encoder input resolution.

Gripper attachments. Three end-effector configurations are used across tasks: the bare parallel-jaw gripper for the two conveyor pick-and-place tasks and the rolling-ball task, a ping-pong paddle mounted between the gripper fingers for the paddle-hitting task, and a small soft net mounted to the gripper for the projectile-catching task.

Projectile launcher. A spring-loaded mechanical projectile launcher placed 2 m from the robot base, with launch angle and velocity manually varied between trials. The launcher fires soft foam balls.

Conveyor.

Conveyor. A custom belt conveyor with speed adjustable from 0 to 25 cm/s in 5 cm/s increments. The 25 cm/s ceiling is a hardware limit of the physical conveyor; the simulation speed sweep extends to 40 cm/s.

Physical setup visualization. Figure 7 shows representative frames from each of the four physical tasks.



(a) Static box, moving obj. (b) Static obj., moving box (c) Paddle hitting (d) Catch a projectile

Figure 7: Representative frames from the physical xArm 7 tasks. Left to right: conveyor pick-and-place variants, paddle-based projectile interception, projectile catching with a soft net mounted to the gripper.

C.3 Baseline Implementation Details

Open-loop VLA. OpenVLA [1] is queried once on the initial observation, and the predicted action sequence is executed open-loop until completion. No further observations are consumed.

Retargeting VLA (closed-loop). OpenVLA is re-queried at every 30 Hz control step with the latest observation. The first action token from each query is executed before the next query begins, in a fully reactive mode without action chunking.

VLA + Fast Replan. OpenVLA with action chunking following the OpenVLA-OFT design [43], with chunk size $K{=}8$ and re-planning at the end of each chunk. The chunk size and replanning interval are tuned on a held-out set of training scenarios to maximize success on dynamic tasks.

Realtime ACT. The Action Chunking Transformer [7] adapted for real-time execution. The encoder consumes the current image and proprioception; the decoder predicts a chunk of 32 future actions. Temporal ensembling averages overlapping predictions from successive chunks. Re-planning occurs every 8 control steps.

Streaming Diffusion Policy. Diffusion Policy [8] with 10 denoising steps and streaming action-chunk re-planning every 4 control steps.

DreamVLA. Zhang et al. [44] introduce a VLA framework that predicts comprehensive world knowledge as intermediate tokens before generating actions. Their model jointly forecasts dynamic regions, depth maps, and semantic features (extracted from DINOv2 and SAM) using a block-wise structured attention mechanism that prevents information leakage between modalities. We use the released DreamVLA implementation and weights (built on OpenVLA-OFT), fine-tuned on our scenario distribution with the same fine-tuning budget as our other baselines.

Track-and-Intercept VLA. A classical motion-informed baseline in which a tracker follows the target object's centroid and fits a constant-acceleration model to its recent positions. The centroid is extrapolated over the same horizon AHEAD realizes, and the current frame is warped so the object appears at its extrapolated location before the frame enters the same frozen OpenVLA. This is a pixel-space analogue of our latent splice. The tracker is CoTracker3, initialized on the largest connected region whose RAFT flow magnitude exceeds τ_{flow}. The constant-acceleration fit is a least-squares fit over the last 5 frames. In multi-object scenes the tracker follows that same region regardless of the instruction, so it has no language grounding.

LaDi-WM (OpenVLA feature space). We retrain the LaDi-WM [16] latent diffusion world model to predict in OpenVLA feature space, replacing its original feature encoders with the frozen OpenVLA vision encoder ϕ. It predicts all $N{=}196$ patch tokens at a fixed horizon of $K{=}5$ steps with 20 denoising steps, receives no optical-flow or kinematic conditioning, and feeds the predicted tokens to the same frozen action decoder as AHEAD. Training uses the same pretraining corpus (Table 7) and the same fine-tuning data as our world model, for approximately 96 GPU-hours on the same hardware.

Training data. Simulation demonstrations are scripted rollouts from a privileged-state planner, 500 per scenario. Every baseline is fine-tuned on these demonstrations with tuned hyperparameters and a shared episode schedule per seed (Section C.5). The $\approx$ 200 real trajectories are static teleoperated pick-and-place episodes plus passive conveyor and launcher clips, so grasping is learned on static scenes and prediction alone supplies the dynamics. The world-model pretraining corpus (Table 7) is passive video. DreamVLA is likewise video-pretrained before a matched fine-tuning budget (Section C.6).

Runtime. The six learned baselines run at 12 to 95 ms per action step, below AHEAD's ≈ 158 ms, so none is compute-disadvantaged and the reactive ones act on fresher frames.

Hyperparameters. Each baseline's training and inference hyperparameters are tuned on a held-out set of 20 training scenarios drawn from the same scenario distribution. The reported numbers use the best-performing hyperparameter configuration per baseline. We report the same fine-tuning protocol (5 epochs on the $\approx$ 200 xArm 7 lab sequences) for all baselines on physical tasks.

C.4 Success Criteria

Each scenario has scenario-specific pass/fail criteria, evaluated automatically in simulation and by hand on the physical robot.

Pick-and-place scenarios (conveyor + cup, beam + cup, pole push + cup, MO, L+MO). Success requires the gripper to close on the target object with all fingers in contact, lift the object by at least 10 cm, and hold it for 3 seconds without dropping. Grasping the wrong object in MO or L+MO scenarios counts as a failure even if the grasp itself succeeds.

Rolling ball. Success requires the gripper to physically stop the ball, with the ball's speed falling below 10% of its pre-contact speed within 500 ms of first contact.

Air hockey. Success requires the puck to be deflected back after being struck by the paddle.

Ballistic catch and projectile catch. Success requires the ball to be inside the net at the moment its vertical velocity first becomes positive (i.e., the ball has been caught before it bounces off the gripper or the floor).

Place in Box (PiB). Success requires the object to be inside the box boundary when the box leaves the workspace, with the object not having touched the table at any point.

Occlusion (Occ). Success requires the gripper to catch the target object after the object reappears from behind the occluder.

Mid-flight trajectory change (Mid). Success requires the ball to be inside the net at the moment its vertical velocity first becomes positive, where the catch must occur after the trajectory deflection. Trials in which the catch completes before the deflection are excluded from the count.

Stress tests. Each stress-test scenario reports success as a function of available reaction time (time-to-exit window). Success requires the gripper to be in contact with the ball within the specified window.

Physical tasks. The first conveyor task (moving ducky, static box) requires picking the ducky off the moving conveyor and placing it in the static box. The second conveyor task (static ducky, moving box) requires picking the static ducky and placing it in the box as the box moves along the conveyor, with the ducky landing inside as the box moves the intercept location. The rolling-ball task requires the gripper to stop the ball before it falls off the table edge. The paddle-hitting task requires deflecting the projectile so that it travels at least 30 cm away from the robot after contact. Re-scoring all trials under the quantitative rolling-ball and paddle criteria changes no outcome. The projectile-catching task requires the ball to remain inside the net after first contact, without bouncing out.

C.5 Statistical Protocol

Simulation. Each cell in the simulation results tables reports the mean success rate across 5 independent training seeds, with 100 evaluation rollouts per seed, for a total of 500 rollouts per cell. Standard error is computed across the 5 seed means rather than across the 500 individual rollouts, so the reported $\pm$ values reflect seed-level variability. All baselines and AHEAD use the same set of 5 seeds.

Physical robot. Each physical task reports successful trials out of 30 attempts. The physical speed-sensitivity sweep reports 10 attempts per speed point conducted in a single session per method to control for between-session drift.

Trial randomization. For each scenario, the per-trial randomization (object positions, motion parameters, lighting in simulation; launcher angle and velocity on hardware) is drawn from a fixed seed schedule shared across all methods, so all methods see the same set of episode instantiations within a seed. This makes the standard errors directly comparable across methods rather than reflecting differences in episode difficulty.

C.6 Compute Resources

Training. World model pretraining ran on $8\times$ NVIDIA H100 80GB GPUs for 200K gradient steps, totaling approximately 96 GPU-hours. The feature alignment layer was trained separately on $1\times$ H100 for approximately 4 GPU-hours. Fine-tuning on the xArm 7 lab sequences ran on $1\times$ H100 for approximately 2 GPU-hours.

Baseline training. Each baseline was trained on the same hardware for a comparable number of total GPU-hours. Fine-tuning DreamVLA on our scenario distribution used approximately 80 GPU-hours, against approximately 102 GPU-hours in total for AHEAD; ACT and Streaming Diffusion Policy fine-tuning used approximately 8 GPU-hours per task suite.

D Additional Results

This appendix evaluates AHEAD under stress-test conditions that exceed the operating envelope of the predict-then-act framework.

D.1 Stress Tests Under Chaotic Dynamics

The Problem Statement's low-order kinematic motion assumption (Section 3) explicitly excludes highly chaotic post-collision dynamics from the operating envelope. This appendix nonetheless evaluates AHEAD under increasingly chaotic conditions to characterize how the predict-then-act framework degrades when its assumptions are violated. Three stress tests are run, each parameterized by the time-to-exit window during which the robot must intercept the moving object: multiple deflection (a ball bouncing off intermediate surfaces), occlusion deflection (deflection combined with target occlusion), and Plinko drop (a ball falling through a peg array, with the most chaotic dynamics).

Table 9: Task success rate (%) on the Multiple Deflection stress test, parameterized by available time-to-exit window. Mean ± standard error over 5 seeds with 100 rollouts each.

Method	100 ms	200 ms	400 ms	800 ms	1600 ms
Open-loop VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.6 ± 1.1	2.4 ± 2.0
VLA + Fast Replan	0.0 ± 0.0	0.0 ± 0.0	1.2 ± 1.5	4.8 ± 2.4	9.2 ± 2.8
Realtime ACT	0.0 ± 0.0	8.6 ± 2.7	22.4 ± 4.1	30.4 ± 4.4	35.6 ± 4.6
Streaming Diffusion Policy	0.0 ± 0.0	0.0 ± 0.0	5.4 ± 2.4	12.8 ± 3.2	18.6 ± 3.7
DreamVLA	0.0 ± 0.0	0.0 ± 0.0	8.6 ± 2.8	18.4 ± 3.8	26.8 ± 4.2
AHEAD (ours)	0.0 ± 0.0	$\mathbf{11.4 \pm 3.0}$	$\mathbf{35.2 \pm 4.5}$	$\mathbf{58.6 \pm 4.6}$	$\mathbf{76.4 \pm 4.1}$

Table 10: Task success rate (%) on the Occlusion Deflection stress test, parameterized by available time-to-exit window. Mean ± standard error over 5 seeds with 100 rollouts each.

Method	100 ms	200 ms	400 ms	800 ms	1600 ms
Open-loop VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.4 ± 0.9	1.8 ± 1.7
VLA + Fast Replan	0.0 ± 0.0	0.0 ± 0.0	0.6 ± 1.1	3.4 ± 2.1	7.2 ± 2.7
Realtime ACT	0.0 ± 0.0	6.4 ± 2.5	16.2 ± 3.6	21.6 ± 4.0	27.4 ± 4.3
Streaming Diffusion Policy	0.0 ± 0.0	0.0 ± 0.0	3.8 ± 2.9	9.4 ± 2.8	14.9 ± 3.4
DreamVLA	0.0 ± 0.0	0.0 ± 0.0	5.6 ± 2.5	13.8 ± 3.4	21.4 ± 4.0
AHEAD (ours)	0.0 ± 0.0	$\mathbf{10.6 \pm 2.9}$	$\mathbf{30.4 \pm 4.4}$	$\mathbf{52.8 \pm 4.7}$	$\mathbf{68.2 \pm 4.4}$

Table 11: Task success rate (%) on the Plinko Drop stress test, parameterized by available time-to-exit window. This scenario explicitly violates the constant-acceleration motion assumption (Section 3) and is included to characterize graceful degradation. Mean ± standard error over 5 seeds with 100 rollouts each.

Method	100 ms	200 ms	400 ms	800 ms	1600 ms
Open-loop VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0
Retargeting VLA	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.6 ± 1.1	1.4 ± 1.5
VLA + Fast Replan	0.0 ± 0.0	0.0 ± 0.0	0.4 ± 0.9	1.8 ± 1.7	4.2 ± 2.3
Realtime ACT	0.0 ± 0.0	6.4 ± 2.5	3.6 ± 2.9	10.4 ± 3.3	17.8 ± 3.8
Streaming Diffusion Policy	0.0 ± 0.0	0.0 ± 0.0	0.6 ± 1.1	3.2 ± 2.0	8.4 ± 2.7
DreamVLA	0.0 ± 0.0	0.0 ± 0.0	1.8 ± 1.7	6.4 ± 2.5	12.6 ± 3.2
AHEAD (ours)	0.0 ± 0.0	$\mathbf{9.8 \pm 2.9}$	$\mathbf{22.6 \pm 4.1}$	$\mathbf{38.4 \pm 4.6}$	$\mathbf{48.6 \pm 4.7}$

Tables 9 through 11 report task success as a function of available reaction time on the three stress scenarios. At the longest time-to-exit window (1600 ms), AHEAD achieves 76.4% on multiple deflection, 68.2% on occlusion deflection, and 48.6% on Plinko drop. The Plinko absolute performance is lower than on any other benchmark in this paper, consistent with the fact that Plinko's

post-collision dynamics are fundamentally chaotic and violate the low-order kinematic motion assumption. The trajectory after each peg contact is highly sensitive to the contact location and angle, so finite-differenced acceleration provides little predictive signal for steps beyond the next collision. Even in this regime, AHEAD's relative advantage over baselines is preserved or amplified. At 1600 ms, AHEAD achieves $2.7\times$ the success of the strongest baseline on Plinko (48.6% vs. 17.8%), $2.1\times$ on multi-deflection, and $2.5\times$ on occlusion deflection. These stress tests bound the operating envelope of predict-then-act control. The claim is graceful degradation under chaotic dynamics, not competitive absolute performance. Adaptive horizon halting contributes to this graceful degradation. The per-scene uncertainty signal truncates rollouts after a peg collision, and Appendix E.4 compares adaptive against fixed horizon on these scenarios (Table 16).

E Ablation Details

This appendix expands the summary in Table 5 with per-scenario results. Each table reports mean $\pm$ standard error over 5 seeds with 100 rollouts each.

E.1 Motion Estimator

Table 12 compares RAFT against three alternative motion estimators and a no-velocity baseline on constant-velocity scenes. The implicit-context row feeds the world model the VLA tokens of the same three frames with no flow input and matched capacity. It falls to 58.6% at 40 cm/s on conveyor + cup, where AHEAD holds 95.4%. Sub-patch displacements are a noisy motion signal in pooled semantic tokens, and without explicit velocity the analytic update of Equation (3) has nothing to propagate.

Table 12: Motion estimator ablation on constant-velocity scenes. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Motion estimator	Conveyor + cup	Beam + cup	Pole push + cup	Rolling ball
RAFT (ours)	$\mathbf{97.3 \pm 1.5}$	$\mathbf{91.7 \pm 2.8}$	$\mathbf{95.0 \pm 2.3}$	$\mathbf{90.0 \pm 3.1}$
CoTracker3	89.3 ± 3.1	87.7 ± 3.3	80.7 ± 4.0	78.3 ± 4.1
Implicit 3-frame context	80.4 ± 3.9	75.6 ± 2.2	73.2 ± 4.4	67.6 ± 3.6
Farnebäck	79.7 ± 4.0	69.3 ± 4.6	68.7 ± 4.7	58.0 ± 4.9
Frame differencing	45.0 ± 5.0	39.7 ± 4.9	40.3 ± 2.1	29.0 ± 4.5
No velocity input	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0	0.0 ± 0.0

E.2 Velocity Model

Table 13 ablates the choice of velocity-update model during multi-step rollout on acceleration/deceleration scenes. The constant-velocity assumption performs poorly on conveyor and pole-push scenes (51% and 58%) where objects accelerate or decelerate, but is sufficient for rolling ball under gravity (91%) where dynamics happen to be well-approximated by initial velocity over short rollouts. A fully learned dynamics model yields competitive performance on three scenarios but collapses on rolling ball (57%), where the gravitational acceleration component is most pronounced.

Table 13: Velocity model ablation on acceleration/deceleration scenes. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Velocity model	Conveyor + cup	Beam + cup	Pole push + cup	Rolling (grav.)
Constant: $V_k = V_0$	50.6 ± 4.3	87.2 ± 2.8	58.4 ± 4.7	90.6 ± 2.4
Exponential decay: $V_k = V_0 e^{-\lambda k}$	42.8 ± 5.1	78.4 ± 3.6	50.2 ± 4.4	89.8 ± 2.6
Kinematic only (no accel in token)	78.2 ± 3.9	86.8 ± 3.4	74.6 ± 4.2	90.4 ± 2.7
Accel in token only (no kinematic)	68.4 ± 4.5	85.4 ± 3.5	68.8 ± 4.6	91.2 ± 2.5
Kinematic + accel-conditioned (ours)	$\mathbf{88.4 \pm 3.1}$	$\mathbf{91.6 \pm 2.6}$	$\mathbf{93.2 \pm 2.3}$	89.4 ± 2.9
Learned dynamics (accel and velocity)	76.8 ± 4.0	89.4 ± 3.0	91.8 ± 2.7	56.8 ± 4.8

E.3 Spatial Masking

Table 14 ablates the language-and-motion saliency masking strategy on constant-velocity scenes. Full prediction is within error of saliency masking in accuracy, so the mask is accuracy-neutral here. Its contribution is latency. Predicting all 196 tokens instead of the 30 to 60 selected exceeds the ≈ 200 ms budget, and the mask keeps prediction deployable.

Table 14: Spatial masking ablation on constant-velocity scenes, with end-to-end latency per action step. Full prediction matches saliency masking in accuracy but exceeds the $\approx$ 200 ms budget. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Masking strategy	Conveyor	Beam	Pole push	Rolling ball	Latency (ms)
Language + motion (ours)	$\mathbf{97.3 \pm 1.5}$	$\mathbf{92.4 \pm 2.6}$	94.6 ± 2.4	$\mathbf{91.2 \pm 2.8}$	158 ± 2
Motion saliency only (no language)	86.2 ± 3.4	79.8 ± 3.9	84.4 ± 3.5	85.6 ± 3.4	151 ± 2
Language only (no motion boost)	87.4 ± 3.2	68.6 ± 4.5	79.2 ± 3.8	76.4 ± 4.1	154 ± 2
Uniform mask (random 30%)	49.4 ± 4.6	41.6 ± 4.8	51.8 ± 4.5	42.8 ± 4.7	160 ± 2
Full prediction (all 196 tokens)	96.0 ± 2.2	86.8 ± 3.3	$\mathbf{95.6 \pm 2.4}$	86.4 ± 3.4	250 ± 3

E.4 Fixed Horizon Sweep

Table 15 reports a controlled fixed-horizon sweep on constant-velocity scenes, characterizing the latency-accuracy tradeoff that motivates AHEAD's adaptive halting mechanism. Outside this subsection, all results use adaptive halting with $K_{\max} = 10$. The optimal fixed horizon varies by scenario. $K = 5$ peaks on conveyor and rolling ball, while $K = 8$ is essentially tied on beam.

Table 15: Fixed-horizon ablation on constant-velocity scenes, characterizing the latency-accuracy tradeoff. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Horizon K	Conveyor + cup	Beam + cup	Pole push + cup	Rolling ball	Latency (ms)
$K = 1$	51.2 ± 4.4	59.8 ± 4.5	48.4 ± 4.7	40.6 ± 4.9	120 ± 1
$K = 2$	58.6 ± 4.5	65.4 ± 4.6	60.2 ± 4.7	56.8 ± 4.8	128 ± 1
$K = 3$	78.6 ± 2.9	84.4 ± 3.6	76.2 ± 4.1	78.8 ± 3.9	138 ± 1
$K = 5$	$\mathbf{96.8 \pm 1.9}$	$\mathbf{92.4 \pm 2.6}$	$\mathbf{94.6 \pm 2.4}$	$\mathbf{91.2 \pm 2.8}$	158 ± 2
$K = 8$	95.6 ± 2.2	91.8 ± 2.8	93.8 ± 2.6	90.4 ± 3.0	185 ± 2
$K = 12$	88.6 ± 3.2	91.4 ± 2.7	87.4 ± 3.4	77.8 ± 4.0	218 ± 3

Fixed K=5 ties adaptive halting on these scenes because halting realizes 3 to 5 steps there. The benefit of adaptivity appears when predictability varies. Table 16 compares the two on the stress tests of Appendix D.1, where post-collision variance truncates rollouts at the contact, and Table 17 reports the realized horizon per scenario group.

Table 16: Adaptive halting versus fixed horizon on the three stress tests at the 1600 ms time-to-exit window. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Horizon	Multiple deflection	Occlusion deflection	Plinko drop
Adaptive halting (ours)	76.4 ± 4.1	68.2 ± 4.4	48.6 ± 4.7
Fixed $K = 5$	63.8 ± 3.5	54.6 ± 6.7	31.4 ± 1.4

E.5 World Model Architecture

Table 18 ablates the number of flow matching samples S and encoder depth on constant-velocity scenes. Reducing S to 1 sample eliminates the trajectory-variance uncertainty signal entirely and drops performance to 58 to 80%. $S = 3$ recovers most of the performance but leaves the uncertainty estimate noisy. $S = 5$ is the best operating point, recovering full performance within the latency budget. $S = 10$ yields marginal further gains at substantially higher compute. A 2-layer encoder is insufficient (70 to 78% success), while 6 layers offer essentially no gain over 4 layers at 14 ms additional cost.

E.6 Token Splicing

Table 19 ablates two aspects of the token splice in Equation (7). Without the alignment layer, Ballistic Catch remains above the best baseline, so the layer, trained once on the pretraining corpus and never per task, recovers fidelity rather than supplying the policy. Excluding robot-arm patches

Table 17: Mean realized rollout horizon K under adaptive halting ($K_{\max} = 10$), averaged over evaluation rollouts.

	Const. vel.	**Accel. / decel.**	**Ballistic**	**Occlusion**	**Plinko**
Realized K (steps)	4.2	3.8	9.1	9.6	1.8

Table 18: World model architecture ablation on constant-velocity scenes. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Configuration	**Conveyor**	**Beam**	**Pole push**	**Rolling ball**	**Latency (ms)**
Flow matching, $S = 1$	78.6 ± 3.8	80.2 ± 3.7	70.4 ± 4.4	58.4 ± 4.6	125 ± 1
Flow matching, $S = 3$	86.4 ± 3.3	88.2 ± 3.1	87.6 ± 3.2	78.4 ± 3.9	140 ± 1
Flow matching, $S = 5$ (ours)	$\mathbf{97.3 \pm 1.5}$	$\mathbf{92.4 \pm 2.6}$	$\mathbf{94.6 \pm 2.4}$	$\mathbf{91.2 \pm 2.8}$	158 ± 2
Flow matching, $S = 10$	97.0 ± 1.8	92.8 ± 2.5	95.0 ± 2.3	91.6 ± 2.7	195 ± 2
Encoder: 2 layers	78.2 ± 3.9	70.4 ± 4.3	68.6 ± 4.5	71.6 ± 4.2	148 ± 2
Encoder: 4 layers (ours)	$\mathbf{96.8 \pm 1.9}$	$\mathbf{92.4 \pm 2.6}$	$\mathbf{94.6 \pm 2.4}$	$\mathbf{91.2 \pm 2.8}$	158 ± 2
Encoder: 6 layers	97.2 ± 1.7	92.6 ± 2.5	95.4 ± 2.2	92.0 ± 2.6	172 ± 2

from $\mathcal{S}$ removes predictions of robot-induced motion that the world model cannot make without action conditioning (Appendix B.4).

Table 19: Token splicing ablation. Removing the feature alignment layer g_{align} tests whether alignment, rather than prediction, supplies the policy. Excluding robot-arm patches from the selected set $\mathcal{S}$ tests the effect of predicting robot-induced motion without action conditioning. Mean $\pm$ standard error over 5 seeds with 100 rollouts each.

Configuration	**Conveyor + cup**	**Ballistic Catch**
AHEAD (default)	97.3 ± 1.5	87.4 ± 3.2
Without alignment layer g_{align}	88.4 ± 3.1	61.1 ± 4.6
Robot-arm patches excluded from $\mathcal{S}$	97.6 ± 1.4	92.2 ± 2.7
Best learned baseline (DreamVLA)	58.3 ± 4.7	49.1 ± 9.1